Aim: To Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch. Document the distinct features and functionality of the packages.

[13]:
```
import numpy as np
```

[14]:
```
!pip install tensorflow
import tensorflow as tf #Tensorflow
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: tensorflow in /home/sapatevaibhav/.local/lib/python3.10/site-packages (2.14.0)
Requirement already satisfied: absl-py>=1.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.0.0)
Requirement already satisfied: astunparse>=1.6.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (23.5.26)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.5.4)
Requirement already satisfied: google-pasta>=0.1.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=2.9.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (3.9.0)
Requirement already satisfied: libclang>=13.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (16.0.6)
Requirement already satisfied: ml-dtypes==0.2.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: numpy>=1.23.5 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.24.0)
Requirement already satisfied: opt-einsum>=2.3.2 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: packaging in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (23.2)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in /home/sapatevaibhav/.local/lib/python
3.10/site-packages (from tensorflow) (4.24.4)
Requirement already satisfied: setuptools in /usr/lib/python3/dist-packages (from tensorflow) (59.6.0)
Requirement already satisfied: six>=1.12.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt<1.15,>=1.11.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.14.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.34.
0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.59.0)
Requirement already satisfied: tensorboard<2.15,>=2.14 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.1)
Requirement already satisfied: tensorflow-estimator<2.15,>=2.14.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.0)
Requirement already satisfied: keras<2.15,>=2.14.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/lib/python3/dist-packages (from astunparse>=1.6.0->tensorflow) (0.37.1)
Requirement already satisfied: google-auth<3,>=1.6.3 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->tensorflo
w) (2.23.2)
Requirement already satisfied: google-auth-oauthlib<1.1,>=0.5 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->
tensorflow) (1.0.0)
Requirement already satisfied: markdown>=2.6.8 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->tensorflow) (3.
4.4)
Requirement already satisfied: requests<3,>=2.21.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->tensorflow)
(2.32.3)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>
=2.14->tensorflow) (0.7.1)
Requirement already satisfied: werkzeug>=1.0.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->tensorflow) (3.
0.2)
Requirement already satisfied: cachetools<6.0,>=2.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from google-auth<3,>=1.6.3->tensorboar
d<2.15,>=2.14->tensorflow) (5.3.1)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from google-auth<3,>=1.6.3->tensorboard
<2.15,>=2.14->tensorflow) (0.3.0)
Requirement already satisfied: rsa<5,>=3.1.4 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from google-auth<3,>=1.6.3->tensorboard<2.15,>=
2.14->tensorflow) (4.9)
Requirement already satisfied: requests-oauthlib>=0.7.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from google-auth-oauthlib<1.1,>=0.5-
>tensorboard<2.15,>=2.14->tensorflow) (1.3.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorboar
d<2.15,>=2.14->tensorflow) (3.2.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/lib/python3/dist-packages (from requests<3,>=2.21.0->tensorboard<2.15,>=2.14->tensorflow) (3.3)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from requests<3,>=2.21.0->tensorboard<2.1
5,>=2.14->tensorflow) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/lib/python3/dist-packages (from requests<3,>=2.21.0->tensorboard<2.15,>=2.14->tensorflow) (202
0.6.20)
Requirement already satisfied: MarkupSafe>=2.1.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from werkzeug>=1.0.1->tensorboard<2.15,>=2.
14->tensorflow) (2.1.5)
Requirement already satisfied: pyasn1<0.6.0,>=0.4.6 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from pyasn1-modules>=0.2.1->google-auth<
```

[17]:
```
(10000, 28, 28)
```

[18]:
```
#4. PyTorch
!pip3 install torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu115
```

```
Defaulting to user installation because normal site-packages is not writeable
Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cu115
Requirement already satisfied: torch in /home/sapatevaibhav/.local/lib/python3.10/site-packages (2.4.1)
Requirement already satisfied: torchvision in /home/sapatevaibhav/.local/lib/python3.10/site-packages (0.19.1)
Requirement already satisfied: torchaudio in /home/sapatevaibhav/.local/lib/python3.10/site-packages (2.4.1)
Requirement already satisfied: filelock in /usr/lib/python3/dist-packages (from torch) (3.6.0)
Requirement already satisfied: typing-extensions>=4.8.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (4.12.2)
Requirement already satisfied: sympy in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (1.13.3)
Requirement already satisfied: networkx in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (3.3)
Requirement already satisfied: jinja2 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (3.1.3)
Requirement already satisfied: fsspec in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (2024.3.1)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.1.105 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.1.105 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.1.105 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.105)
Requirement already satisfied: nvidia-cudnn-cu12==9.1.0.70 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (9.1.0.70)
Requirement already satisfied: nvidia-cublas-cu12==12.1.3.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.3.1)
Requirement already satisfied: nvidia-cufft-cu12==11.0.2.54 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (11.0.2.54)
Requirement already satisfied: nvidia-curand-cu12==10.3.2.106 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (10.3.2.106)
Requirement already satisfied: nvidia-cusolver-cu12==11.4.5.107 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (11.4.5.107)
Requirement already satisfied: nvidia-cusparse-cu12==12.1.0.106 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.0.106)
Requirement already satisfied: nvidia-nccl-cu12==2.20.5 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (2.20.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.1.105 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (12.1.105)
Requirement already satisfied: triton==3.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torch) (3.0.0)
Requirement already satisfied: nvidia-nvjitlink-cu12 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from nvidia-cusolver-cu12==11.4.5.107->
torch) (12.6.77)
Requirement already satisfied: numpy in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from torchvision) (1.24.0)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/lib/python3/dist-packages (from torchvision) (9.0.1)
Requirement already satisfied: MarkupSafe>=2.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from jinja2->torch) (2.1.5)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from sympy->torch) (1.3.0)
```

[19]:
```
pip install --upgrade pip
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: pip in /home/sapatevaibhav/.local/lib/python3.10/site-packages (24.2)
Note: you may need to restart the kernel to use updated packages.
```

[20]:
```
import torch
import torch.nn as nn
```

[21]:
```
print(torch.__version__)
```

```
2.4.1+cu121
```

[22]:
```
torch.cuda.is_available()
```

[22]:
```
False
```

Feature	TensorFlow	Keras	Theano	PyTorch
Interface	C++, Python, CUDA	Python	Python	Python
Architecture	Complex static graphs	Simple high-level API	Static graphs	Dynamic graphs
Performance	High performance	Depends on backend	High performance	Competitive
Learning Curve	Steep	Easy	Moderate	Gentle
Community Support	Extensive	Growing	Limited	Increasing
Use Case Suitability	Large datasets	Small datasets	Academic research	Research and prototyping

```
Aim: To Implement Feedforward neural networks with Keras and TensorFlow
a. Import the necessary packages
b. Load the training and testing data (MNIST/CIFAR10)
c. Define the network architecture using Keras
d. Train the model using SGD
e. Evaluate the network
f. Plot the training loss and accuracy
```

[2]:
```
#installations
!pip install scikit-learn tensorflow matplotlib numpy
from sklearn.preprocessing import LabelBinarizer
from sklearn.metrics import classification_report
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import matplotlib.pyplot as plt
import numpy as np
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: scikit-learn in /home/sapatevaibhav/.local/lib/python3.10/site-packages (1.1.2)
Requirement already satisfied: tensorflow in /home/sapatevaibhav/.local/lib/python3.10/site-packages (2.14.0)
Requirement already satisfied: matplotlib in /home/sapatevaibhav/.local/lib/python3.10/site-packages (3.6.0)
Requirement already satisfied: numpy in /home/sapatevaibhav/.local/lib/python3.10/site-packages (1.24.0)
Requirement already satisfied: scipy>=1.3.2 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from scikit-learn) (1.9.1)
Requirement already satisfied: joblib>=1.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from scikit-learn) (1.2.0)
Requirement already satisfied: threadpoolctl>=2.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from scikit-learn) (3.1.0)
Requirement already satisfied: absl-py>=1.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.0.0)
Requirement already satisfied: astunparse>=1.6.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (23.5.26)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.5.4)
Requirement already satisfied: google-pasta>=0.1.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=2.9.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (3.9.0)
Requirement already satisfied: libclang>=13.0.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (16.0.6)
Requirement already satisfied: ml-dtypes==0.2.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: opt-einsum>=2.3.2 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: packaging in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (23.2)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in /home/sapatevaibhav/.local/lib/python
3.10/site-packages (from tensorflow) (4.24.4)
Requirement already satisfied: setuptools in /usr/lib/python3/dist-packages (from tensorflow) (59.6.0)
Requirement already satisfied: six>=1.12.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt<1.15,>=1.11.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.14.1)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (0.34.
0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (1.59.0)
Requirement already satisfied: tensorboard<2.15,>=2.14 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.1)
Requirement already satisfied: tensorflow-estimator<2.15,>=2.14.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.0)
Requirement already satisfied: keras<2.15,>=2.14.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorflow) (2.14.0)
Requirement already satisfied: contourpy>=1.0.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from matplotlib) (1.0.5)
Requirement already satisfied: cycler>=0.10 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from matplotlib) (4.37.4)
Requirement already satisfied: kiwisolver>=1.0.1 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from matplotlib) (1.4.4)
Requirement already satisfied: pillow>=6.2.0 in /usr/lib/python3/dist-packages (from matplotlib) (9.0.1)
Requirement already satisfied: pyparsing>=2.2.1 in /usr/lib/python3/dist-packages (from matplotlib) (2.4.7)
Requirement already satisfied: python-dateutil>=2.7 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/lib/python3/dist-packages (from astunparse>=1.6.0->tensorflow) (0.37.1)
Requirement already satisfied: google-auth<3,>=1.6.3 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->tensorflo
w) (2.23.2)
Requirement already satisfied: google-auth-oauthlib<1.1,>=0.5 in /home/sapatevaibhav/.local/lib/python3.10/site-packages (from tensorboard<2.15,>=2.14->
```

```
X_test = X_test.reshape((X_test.shape[0], 28 * 28 * 1))
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0
```

[4]:
```
lb = LabelBinarizer()
Y_train = lb.fit_transform(Y_train)
Y_test = lb.transform(Y_test)
```

[5]:
```
#building the model
model = Sequential()
model.add(Dense(128, input_shape=(784,), activation="sigmoid"))
model.add(Dense(64, activation="sigmoid"))
model.add(Dense(10, activation="softmax"))
```

[6]:
```
sgd = SGD(0.01)
epochs=10
model.compile(loss="categorical_crossentropy", optimizer=sgd,metrics=["accuracy"])
H = model.fit(X_train, Y_train, validation_data=(X_test, Y_test),epochs=epochs, batch_size=128)
```

```
2024-10-02 23:01:24.986785: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 188160000 exceeds 10% of free system memory.
Epoch 1/10
468/469 [============================>.] - ETA: 0s - loss: 2.2862 - accuracy: 0.1459
2024-10-02 23:01:32.506051: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 31360000 exceeds 10% of free system memory.
469/469 [==============================] - 7s 13ms/step - loss: 2.2862 - accuracy: 0.1461 - val_loss: 2.2556 - val_accuracy: 0.2173
Epoch 2/10
469/469 [==============================] - 5s 12ms/step - loss: 2.2300 - accuracy: 0.3415 - val_loss: 2.2011 - val_accuracy: 0.4223
Epoch 3/10
469/469 [==============================] - 5s 11ms/step - loss: 2.1678 - accuracy: 0.4918 - val_loss: 2.1277 - val_accuracy: 0.4570
Epoch 4/10
469/469 [==============================] - 5s 10ms/step - loss: 2.0812 - accuracy: 0.5401 - val_loss: 2.0230 - val_accuracy: 0.5543
Epoch 5/10
469/469 [==============================] - 5s 10ms/step - loss: 1.9576 - accuracy: 0.5811 - val_loss: 1.8761 - val_accuracy: 0.6276
Epoch 6/10
469/469 [==============================] - 6s 12ms/step - loss: 1.7920 - accuracy: 0.6148 - val_loss: 1.6905 - val_accuracy: 0.6550
Epoch 7/10
469/469 [==============================] - 5s 11ms/step - loss: 1.5990 - accuracy: 0.6508 - val_loss: 1.4912 - val_accuracy: 0.6827
Epoch 8/10
469/469 [==============================] - 5s 11ms/step - loss: 1.4075 - accuracy: 0.6927 - val_loss: 1.3078 - val_accuracy: 0.7220
Epoch 9/10
469/469 [==============================] - 5s 11ms/step - loss: 1.2401 - accuracy: 0.7300 - val_loss: 1.1544 - val_accuracy: 0.7659
Epoch 10/10
469/469 [==============================] - 5s 10ms/step - loss: 1.1032 - accuracy: 0.7627 - val_loss: 1.0314 - val_accuracy: 0.7757
```

[7]:
```
#making the predictions
predictions = model.predict(X_test, batch_size=128)
print(classification_report(Y_test.argmax(axis=1),predictions.argmax(axis=1),target_names=[str(x) for x in lb.classes_]))
```

```
79/79 [==============================] - 1s 6ms/step
              precision    recall  f1-score   support

           0       0.83      0.97      0.89       980
           1       0.78      0.99      0.87      1135
           2       0.84      0.72      0.77      1032
           3       0.65      0.80      0.72      1010
           4       0.68      0.92      0.78       982
           5       0.87      0.37      0.52       892
           6       0.85      0.89      0.87       958
           7       0.77      0.89      0.82      1028
           8       0.81      0.68      0.74       974
           9       0.85      0.47      0.61      1009

    accuracy                           0.78     10000
   macro avg       0.79      0.77      0.76     10000
weighted avg       0.79      0.78      0.76     10000
```

```
              2       0.84      0.72      0.77      1032
              3       0.65      0.80      0.72      1010
              4       0.68      0.92      0.78       982
              5       0.87      0.37      0.52       892
              6       0.85      0.89      0.87       958
              7       0.77      0.89      0.82      1028
              8       0.81      0.68      0.74       974
              9       0.85      0.47      0.61      1009

       accuracy                           0.78     10000
      macro avg       0.79      0.77      0.76     10000
   weighted avg       0.79      0.78      0.76     10000
```

[8]:
```
#plotting the training loss and accuracy
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, epochs), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, epochs), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, epochs), H.history["accuracy"], label="train_acc")
plt.plot(np.arange(0, epochs), H.history["val_accuracy"], label="val_acc")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend()
```

[8]:
```
<matplotlib.legend.Legend at 0x7f6b78631ed0>
```



Aim: To Build the Image classification model by dividing the model into following 4 stages:
a. Loading and preprocessing the image data
b. Defining the model's architecture
c. Training the model
d. Estimating the model's performance

[1]:
```
#importing the libraries
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
```

```
2024-10-07 02:42:06.774927: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-07 02:42:06.819308: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to register cuDNN factory: Attempting to register fa
ctory for plugin cuDNN when one has already been registered
2024-10-07 02:42:06.819345: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register cuFFT factory: Attempting to register fac
tory for plugin cuFFT when one has already been registered
2024-10-07 02:42:06.819377: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to register cuBLAS factory: Attempting to register
factory for plugin cuBLAS when one has already been registered
2024-10-07 02:42:06.827311: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-07 02:42:06.827631: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions i
n performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-10-07 02:42:08.252365: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
```

[2]:
```
#grabbing CIFAR10 dataset
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()
train_images, test_images = train_images / 255.0, test_images / 255.0
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 [==============================] - 618s 4us/step
```

[3]:
```
#showing images of mentioned categories
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer','dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(10):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



[4]:
```
#building CNN model
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))

model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 30, 30, 32)        896

 max_pooling2d (MaxPooling2  (None, 15, 15, 32)        0
 D)

 conv2d_1 (Conv2D)           (None, 13, 13, 64)        18496

 max_pooling2d_1 (MaxPoolin  (None, 6, 6, 64)          0
 g2D)

 conv2d_2 (Conv2D)           (None, 4, 4, 64)          36928

 flatten (Flatten)           (None, 1024)              0

 dense (Dense)               (None, 64)                65600

 dense_1 (Dense)             (None, 10)                650

=================================================================
Total params: 122570 (478.79 KB)
Trainable params: 122570 (478.79 KB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

[5]:
```
#model compilation
model.compile(optimizer='adam',loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),metrics=['accuracy'])
epochs = 1
h = model.fit(train_images, train_labels, epochs=epochs, validation_data=(test_images, test_labels))
```

```
2024-10-02 23:15:27.100936: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 614400000 exceeds 10% of free system memory.
1563/1563 [==============================] - ETA: 0s - loss: 1.5268 - accuracy: 0.4407
2024-10-02 23:17:50.480513: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 122880000 exceeds 10% of free system memory.
1563/1563 [==============================] - 151s 92ms/step - loss: 1.5268 - accuracy: 0.4407 - val_loss: 1.2683 - val_accuracy: 0.5452
```

[ ]:

Aim: To Use Autoencoder to implement anomaly detection. Build the model by using:
a. Import required libraries
b. Upload / access the dataset
c. Encoder converts it into latent representation
d. Decoder networks convert it back to the original input
e. Compile the models with Optimizer, Loss, and Evaluation Metrics

[1]:
```
#importing Libraries and dataset
import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras import Model, Sequential
from tensorflow.keras.layers import Dense, Dropout
from sklearn.model_selection import train_test_split
from tensorflow.keras.losses import MeanSquaredLogarithmicError

PATH_TO_DATA = 'http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv'
data = pd.read_csv(PATH_TO_DATA, header=None)
data.head()
```

```
2024-10-02 23:06:24.805983: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-02 23:06:24.883987: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to register cuDNN factory: Attempting to register fa
ctory for plugin cuDNN when one has already been registered
2024-10-02 23:06:24.884055: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register cuFFT factory: Attempting to register fac
tory for plugin cuFFT when one has already been registered
2024-10-02 23:06:24.884106: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to register cuBLAS factory: Attempting to register
factory for plugin cuBLAS when one has already been registered
2024-10-02 23:06:24.897962: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-02 23:06:24.900345: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions i
n performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-10-02 23:06:26.549846: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
```

[1]:

	0	1	2	3	4	5	6	7	8	9	...	131	132	133	134	135	
0	-0.112522	-2.827204	-3.773897	-4.349751	-4.376041	-3.474986	-2.181408	-1.818286	-1.250522	-0.477492	...	0.792168	0.933541	0.796958	0.578621	0.257740	0
1	-1.100878	-3.996840	-4.285843	-4.506579	-4.022377	-3.234368	-1.566126	-0.992258	-0.754680	0.042321	...	0.538356	0.656881	0.787490	0.724046	0.555784	0
2	-0.567088	-2.593450	-3.874230	-4.584095	-4.187449	-3.151462	-1.742940	-1.490659	-1.183580	-0.394229	...	0.886073	0.531452	0.311377	-0.021919	-0.713683	-0
3	0.490473	-1.914407	-3.616364	-4.318823	-4.268016	-3.881110	-2.993280	-1.671131	-1.333884	-0.965629	...	0.350816	0.499111	0.600345	0.842069	0.952074	0
4	0.800232	-0.874252	-2.384761	-3.973292	-4.338224	-3.802422	-2.534510	-1.783423	-1.594450	-0.753199	...	1.148884	0.958434	1.059025	1.371682	1.277392	0

5 rows × 141 columns

[2]:
```
#finding shape of the dataset
data.shape
```

[2]: (4998, 141)

[3]:
```
#splitting training and testing dataset
features = data.drop(140, axis=1)
target = data[140]
x_train, x_test, y_train, y_test = train_test_split(
```

[4]:
```
#scaling the data using MinMaxScaler
min_max_scaler = MinMaxScaler(feature_range=(0, 1))
x_train_scaled = min_max_scaler.fit_transform(train_data.copy())
x_test_scaled = min_max_scaler.transform(x_test.copy())
```

[5]:
```
#creating autoencoder subclass by extending Model class from keras
class AutoEncoder(Model):
  def __init__(self, output_units, ldim=8):
    super().__init__()
    self.encoder = Sequential([
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(ldim, activation='relu')
    ])
    self.decoder = Sequential([
      Dense(16, activation='relu'),
      Dropout(0.1),
      Dense(32, activation='relu'),
      Dropout(0.1),
      Dense(64, activation='relu'),
      Dropout(0.1),
      Dense(output_units, activation='sigmoid')
    ])

  def call(self, inputs):
    encoded = self.encoder(inputs)
    decoded = self.decoder(encoded)
    return decoded
```

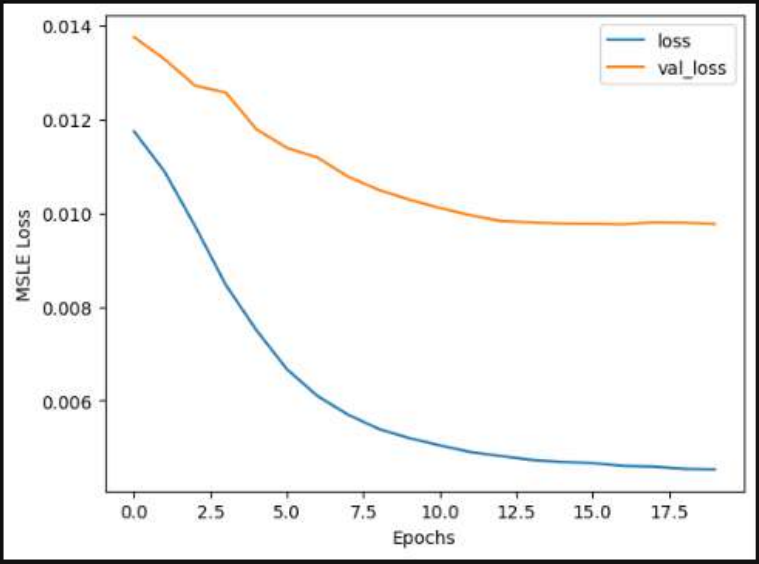
[6]:
```
#model configuration
model = AutoEncoder(output_units=x_train_scaled.shape[1])
model.compile(loss='msle', metrics=['mse'], optimizer='adam')
epochs = 20

history = model.fit(
    x_train_scaled,
    x_train_scaled,
    epochs=epochs,
    batch_size=512,
    validation_data=(x_test_scaled, x_test_scaled)
)
```

```
Epoch 1/20
5/5 [==============================] - 4s 140ms/step - loss: 0.0117 - mse: 0.0267 - val_loss: 0.0138 - val_mse: 0.0324
Epoch 2/20
5/5 [==============================] - 0s 41ms/step - loss: 0.0109 - mse: 0.0248 - val_loss: 0.0133 - val_mse: 0.0313
Epoch 3/20
5/5 [==============================] - 0s 47ms/step - loss: 0.0097 - mse: 0.0221 - val_loss: 0.0127 - val_mse: 0.0299
Epoch 4/20
5/5 [==============================] - 0s 59ms/step - loss: 0.0085 - mse: 0.0191 - val_loss: 0.0126 - val_mse: 0.0294
Epoch 5/20
5/5 [==============================] - 0s 48ms/step - loss: 0.0075 - mse: 0.0169 - val_loss: 0.0118 - val_mse: 0.0276
Epoch 6/20
5/5 [==============================] - 0s 53ms/step - loss: 0.0067 - mse: 0.0150 - val_loss: 0.0114 - val_mse: 0.0267
Epoch 7/20
5/5 [==============================] - 0s 36ms/step - loss: 0.0061 - mse: 0.0137 - val_loss: 0.0112 - val_mse: 0.0262
Epoch 8/20
5/5 [==============================] - 0s 42ms/step - loss: 0.0057 - mse: 0.0128 - val_loss: 0.0108 - val_mse: 0.0253
```

```
Epoch 20/20
5/5 [==============================] - 0s 51ms/step - loss: 0.0045 - mse: 0.0102 - val_loss: 0.0098 - val_mse: 0.0231
```

[7]:
```python
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel('Epochs')
plt.ylabel('MSLE Loss')
plt.legend(['loss', 'val_loss'])
plt.show()
```



[8]:
```python
#finding threshold for anomaly and doing predictions
def find_threshold(model, x_train_scaled):
  reconstructions = model.predict(x_train_scaled)
  reconstruction_errors = tf.keras.losses.msle(reconstructions, x_train_scaled)
  threshold = np.mean(reconstruction_errors.numpy()) \
   + np.std(reconstruction_errors.numpy())
  return threshold

def get_predictions(model, x_test_scaled, threshold):
  predictions = model.predict(x_test_scaled)
  errors = tf.keras.losses.msle(predictions, x_test_scaled)
  anomaly_mask = pd.Series(errors) > threshold
  preds = anomaly_mask.map(lambda x: 0.0 if x == True else 1.0)
  return preds

threshold = find_threshold(model, x_train_scaled)
print(f"Threshold: {threshold}")
```

```
73/73 [==============================] - 0s 3ms/step
Threshold: 0.00972221153034655
```

[9]:
```python
#getting accuracy score
predictions = get_predictions(model, x_test_scaled, threshold)
accuracy_score(predictions, y_test)
```

```
32/32 [==============================] - 0s 6ms/step
```

[9]: 0.942

Aim: To Implement the Continuous Bag of Words (CBOW) Model. Stages can be:
a. Data preparation
b. Generate training data
c. Train model
d. Output

[1]:
```
# Import necessary libraries
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Lambda, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Sample corpus
corpus = [
    "the cat sat on the mat",
    "the dog sat on the log",
    "cats and dogs are great pets",
    "the mat is soft and warm"
]

# Preprocess text: Tokenization and lowercasing
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
total_words = len(tokenizer.word_index) + 1  # +1 for padding

# Convert text to sequences
sequences = tokenizer.texts_to_sequences(corpus)
```

```
2024-10-07 02:48:52.455903: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-07 02:48:52.507897: E tensorflow/compiler/xla/stream_executor/cuda/cuda_dnn.cc:9342] Unable to register cuDNN factory: Attempting to register fa
ctory for plugin cuDNN when one has already been registered
2024-10-07 02:48:52.508009: E tensorflow/compiler/xla/stream_executor/cuda/cuda_fft.cc:609] Unable to register cuFFT factory: Attempting to register fac
tory for plugin cuFFT when one has already been registered
2024-10-07 02:48:52.508136: E tensorflow/compiler/xla/stream_executor/cuda/cuda_blas.cc:1518] Unable to register cuBLAS factory: Attempting to register
factory for plugin cuBLAS when one has already been registered
2024-10-07 02:48:52.517996: I tensorflow/tsl/cuda/cudart_stub.cc:28] Could not find cuda drivers on your machine, GPU will not be used.
2024-10-07 02:48:52.518871: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions i
n performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2024-10-07 02:48:53.923387: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
```

[2]:
```
## Stage b: Generate Training Data
def generate_training_data(sequences, window_size=2):
    contexts = []
    targets = []

    for sequence in sequences:
        for i in range(window_size, len(sequence) - window_size):
            context = sequence[i - window_size:i] + sequence[i + 1:i + window_size + 1]
            target = sequence[i]
            contexts.append(context)
            targets.append(target)

    return np.array(contexts), np.array(targets)

X, y = generate_training_data(sequences)
```

```
    return np.array(contexts), np.array(targets)

X, y = generate_training_data(sequences)

# Pad sequences for consistent input shape
X = pad_sequences(X, maxlen=4)  # Adjust maxlen based on context size
```

[8]:
```
## Stage c: Train Model
# Define CBOW model architecture
model = Sequential()
model.add(Embedding(input_dim=total_words, output_dim=10, input_length=4))
model.add(Lambda(lambda x: tf.reduce_mean(x, axis=1)))  # Average embeddings
model.add(Dense(total_words, activation='softmax'))

# Compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# Train the model
model.fit(X, y, epochs=100)
```

```
Epoch 1/100
1/1 [==============================] - 1s 537ms/step - loss: 2.8329 - accuracy: 0.1250
Epoch 2/100
1/1 [==============================] - 0s 14ms/step - loss: 2.8297 - accuracy: 0.2500
Epoch 3/100
1/1 [==============================] - 0s 20ms/step - loss: 2.8266 - accuracy: 0.3750
Epoch 4/100
1/1 [==============================] - 0s 15ms/step - loss: 2.8234 - accuracy: 0.5000
Epoch 5/100
1/1 [==============================] - 0s 20ms/step - loss: 2.8203 - accuracy: 0.5000
Epoch 6/100
1/1 [==============================] - 0s 12ms/step - loss: 2.8171 - accuracy: 0.5000
Epoch 7/100
1/1 [==============================] - 0s 15ms/step - loss: 2.8140 - accuracy: 0.5000
Epoch 8/100
1/1 [==============================] - 0s 16ms/step - loss: 2.8109 - accuracy: 0.5000
Epoch 9/100
1/1 [==============================] - 0s 21ms/step - loss: 2.8077 - accuracy: 0.5000
Epoch 10/100
1/1 [==============================] - 0s 27ms/step - loss: 2.8046 - accuracy: 0.6250
Epoch 11/100
1/1 [==============================] - 0s 14ms/step - loss: 2.8015 - accuracy: 0.7500
Epoch 12/100
1/1 [==============================] - 0s 19ms/step - loss: 2.7984 - accuracy: 0.7500
Epoch 13/100
1/1 [==============================] - 0s 10ms/step - loss: 2.7952 - accuracy: 0.7500
Epoch 14/100
1/1 [==============================] - 0s 13ms/step - loss: 2.7921 - accuracy: 0.7500
Epoch 15/100
1/1 [==============================] - 0s 11ms/step - loss: 2.7890 - accuracy: 0.8750
Epoch 16/100
1/1 [==============================] - 0s 19ms/step - loss: 2.7858 - accuracy: 0.8750
Epoch 17/100
1/1 [==============================] - 0s 10ms/step - loss: 2.7827 - accuracy: 0.8750
Epoch 18/100
1/1 [==============================] - 0s 20ms/step - loss: 2.7795 - accuracy: 0.8750
Epoch 19/100
1/1 [==============================] - 0s 9ms/step - loss: 2.7763 - accuracy: 0.8750
Epoch 20/100
1/1 [==============================] - 0s 13ms/step - loss: 2.7731 - accuracy: 0.8750
Epoch 21/100
1/1 [==============================] - 0s 16ms/step - loss: 2.7699 - accuracy: 0.8750
Epoch 22/100
```

```
Epoch 92/100
1/1 [==============================] - 0s 13ms/step - loss: 2.4745 - accuracy: 1.0000
Epoch 93/100
1/1 [==============================] - 0s 21ms/step - loss: 2.4691 - accuracy: 1.0000
Epoch 94/100
1/1 [==============================] - 0s 22ms/step - loss: 2.4637 - accuracy: 1.0000
Epoch 95/100
1/1 [==============================] - 0s 19ms/step - loss: 2.4582 - accuracy: 1.0000
Epoch 96/100
1/1 [==============================] - 0s 16ms/step - loss: 2.4527 - accuracy: 1.0000
Epoch 97/100
1/1 [==============================] - 0s 15ms/step - loss: 2.4472 - accuracy: 1.0000
Epoch 98/100
1/1 [==============================] - 0s 16ms/step - loss: 2.4417 - accuracy: 1.0000
Epoch 99/100
1/1 [==============================] - 0s 12ms/step - loss: 2.4361 - accuracy: 1.0000
Epoch 100/100
1/1 [==============================] - 0s 15ms/step - loss: 2.4304 - accuracy: 1.0000
```

[8]:
```
<keras.src.callbacks.History at 0x7f6f80c165c0>
```

[16]:
```
## Stage d: Output
# Get word embeddings from the trained model
word_embeddings = model.layers[0].get_weights()[0]

# Create a mapping of words to their embeddings
word_index = tokenizer.word_index


print('Vocabulary Size:', len(word_index))
print('Vocabulary Sample:', list(word_index.items())[:10],"\n\n")


embeddings_dict = {word: word_embeddings[idx] for word, idx in word_index.items()}

# Output the embeddings for each word in a structured format
print("{:<10} | {}".format("Word", "Embedding"))
print("-" * 40)
for word, embedding in embeddings_dict.items():
    print("{:<10} | {}".format(word, np.round(embedding, 3)))
```

```
Vocabulary Size: 16
Vocabulary Sample: [('the', 1), ('sat', 2), ('on', 3), ('mat', 4), ('and', 5), ('cat', 6), ('dog', 7), ('log', 8), ('cats', 9), ('dogs', 10)]


Word       | Embedding
----------------------------------------
the        | [ 0.107 -0.068  0.133 -0.125 -0.146 -0.064  0.076 -0.07   0.142  0.153]
sat        | [ 0.038 -0.151  0.118  0.161  0.06  -0.075  0.104 -0.105  0.121  0.084]
on         | [ 0.072 -0.111  0.172 -0.084 -0.104 -0.114  0.073 -0.098 -0.087  0.139]
mat        | [ 0.103  0.11  -0.086  0.096 -0.078  0.16   0.119 -0.071  0.068  0.076]
and        | [-0.143 -0.113  0.105  0.156 -0.095  0.115 -0.062 -0.159 -0.112  0.086]
cat        | [ 0.131 -0.106  0.151 -0.074 -0.084 -0.141  0.093 -0.097  0.119  0.159]
dog        | [ 0.145 -0.159  0.165 -0.136 -0.096 -0.056  0.111 -0.126  0.162  0.143]
log        | [ 0.05  -0.104  0.103  0.153  0.104 -0.148  0.153 -0.134  0.14   0.104]
cats       | [-0.148 -0.14   0.139  0.131  0.075  0.154 -0.112  0.126 -0.066  0.052]
dogs       | [-0.154  0.068  0.144  0.067  0.145 -0.141  0.086 -0.101 -0.141 -0.11 ]
are        | [-0.105 -0.139  0.104  0.07   0.139  0.143 -0.1    0.045 -0.127  0.071]
great      | [-0.147 -0.124  0.104  0.157  0.071  0.061 -0.052 -0.165 -0.061 -0.124]
pets       | [-0.128  0.095  0.074  0.064  0.149 -0.11   0.166 -0.074 -0.129 -0.101]
is         | [ 0.081  0.089  0.068 -0.135 -0.089  0.149  0.105 -0.134 -0.094  0.095]
soft       | [ 0.088  0.116 -0.1    0.08  -0.12  -0.049 -0.137 -0.171  0.057  0.061]
warm       | [ 0.117  0.102  0.093 -0.118 -0.156  0.098  0.108 -0.093 -0.065  0.111]
```

[ ]:

[2]:
```
# Import necessary libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
import matplotlib.pyplot as plt
```

[3]:
```
# Generate example data (Replace this with your actual dataset)
num_samples = 1000  # Number of samples
num_classes = 10    # Number of classes
X_train = np.random.rand(num_samples, 224, 224, 3).astype(np.float32)
y_train = np.random.randint(0, num_classes, num_samples)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)
```

```
X_train shape: (1000, 224, 224, 3)
y_train shape: (1000,)
```

[4]:
```
# Load pre-trained MobileNetV2 model + higher-level layers
base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Display the model architecture
base_model.summary()
```

```
Model: "mobilenetv2_1.00_224"
__________________________________________________________________________________________________
 Layer (type)                Output Shape              Param #   Connected to
==================================================================================================
 input_1 (InputLayer)        [(None, 224, 224, 3)]     0         []

 Conv1 (Conv2D)              (None, 112, 112, 32)      864       ['input_1[0][0]']

 bn_Conv1 (BatchNormalizati  (None, 112, 112, 32)      128       ['Conv1[0][0]']
 on)

 Conv1_relu (ReLU)           (None, 112, 112, 32)      0         ['bn_Conv1[0][0]']

 expanded_conv_depthwise (D  (None, 112, 112, 32)      288       ['Conv1_relu[0][0]']
 epthwiseConv2D)

 expanded_conv_depthwise_BN  (None, 112, 112, 32)      128       ['expanded_conv_depthwise[0][0
  (BatchNormalization)                                           ]']
```

[5]:
```
# Freeze the lower convolutional layers
for layer in base_model.layers:
    layer.trainable = False

# Check which layers are frozen
frozen_layers = [layer.name for layer in base_model.layers if not layer.trainable]
print("Frozen layers:", frozen_layers)
```

```
Frozen layers: ['input_1', 'Conv1', 'bn_Conv1', 'Conv1_relu', 'expanded_conv_depthwise', 'expanded_conv_depthwise_BN', 'expanded_conv_depthwise_relu',
'expanded_conv_project', 'expanded_conv_project_BN', 'block_1_expand', 'block_1_expand_BN', 'block_1_expand_relu', 'block_1_pad', 'block_1_depthwise',
'block_1_depthwise_BN', 'block_1_depthwise_relu', 'block_1_project', 'block_1_project_BN', 'block_2_expand', 'block_2_expand_BN', 'block_2_expand_relu',
'block_2_depthwise', 'block_2_depthwise_BN', 'block_2_depthwise_relu', 'block_2_project', 'block_2_project_BN', 'block_2_add', 'block_3_expand', 'block_
3_expand_BN', 'block_3_expand_relu', 'block_3_pad', 'block_3_depthwise', 'block_3_depthwise_BN', 'block_3_depthwise_relu', 'block_3_project', 'block_3_p
roject_BN', 'block_4_expand', 'block_4_expand_BN', 'block_4_expand_relu', 'block_4_depthwise', 'block_4_depthwise_BN', 'block_4_depthwise_relu', 'block_
4_project', 'block_4_project_BN', 'block_4_add', 'block_5_expand', 'block_5_expand_BN', 'block_5_expand_relu', 'block_5_depthwise', 'block_5_depthwise_B
N', 'block_5_depthwise_relu', 'block_5_project', 'block_5_project_BN', 'block_5_add', 'block_6_expand', 'block_6_expand_BN', 'block_6_expand_relu', 'blo
ck_6_pad', 'block_6_depthwise', 'block_6_depthwise_BN', 'block_6_depthwise_relu', 'block_6_project', 'block_6_project_BN', 'block_7_expand', 'block_7_ex
```

```
k_10_depthwise', 'block_10_depthwise_BN', 'block_10_depthwise_relu', 'block_10_project', 'block_10_project_BN', 'block_11_expand', 'block_11_expand_BN',
'block_11_expand_relu', 'block_11_depthwise', 'block_11_depthwise_BN', 'block_11_depthwise_relu', 'block_11_project', 'block_11_project_BN', 'block_11_a
dd', 'block_12_expand', 'block_12_expand_BN', 'block_12_expand_relu', 'block_12_depthwise', 'block_12_depthwise_BN', 'block_12_depthwise_relu', 'block_1
2_project', 'block_12_project_BN', 'block_12_add', 'block_13_expand', 'block_13_expand_BN', 'block_13_expand_relu', 'block_13_pad', 'block_13_depthwis
e', 'block_13_depthwise_BN', 'block_13_depthwise_relu', 'block_13_project', 'block_13_project_BN', 'block_14_expand', 'block_14_expand_BN', 'block_14_ex
pand_relu', 'block_14_depthwise', 'block_14_depthwise_BN', 'block_14_depthwise_relu', 'block_14_project', 'block_14_project_BN', 'block_14_add', 'block_
15_expand', 'block_15_expand_BN', 'block_15_expand_relu', 'block_15_depthwise', 'block_15_depthwise_BN', 'block_15_depthwise_relu', 'block_15_project',
'block_15_project_BN', 'block_15_add', 'block_16_expand', 'block_16_expand_BN', 'block_16_expand_relu', 'block_16_depthwise', 'block_16_depthwise_BN',
'block_16_depthwise_relu', 'block_16_project', 'block_16_project_BN', 'Conv_1', 'Conv_1_bn', 'out_relu']
```

[6]:
```
# Create a new model with the base and custom classifier
model = Sequential()
model.add(base_model)
model.add(GlobalAveragePooling2D())
model.add(Dense(256, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))  # Adjust the number of classes as needed

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

# Display the complete model architecture
model.summary()
```

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 mobilenetv2_1.00_224 (Func  (None, 7, 7, 1280)        2257984
 tional)

 global_average_pooling2d (  (None, 1280)              0
 GlobalAveragePooling2D)

 dense (Dense)               (None, 256)               327936

 dense_1 (Dense)             (None, 10)                2570

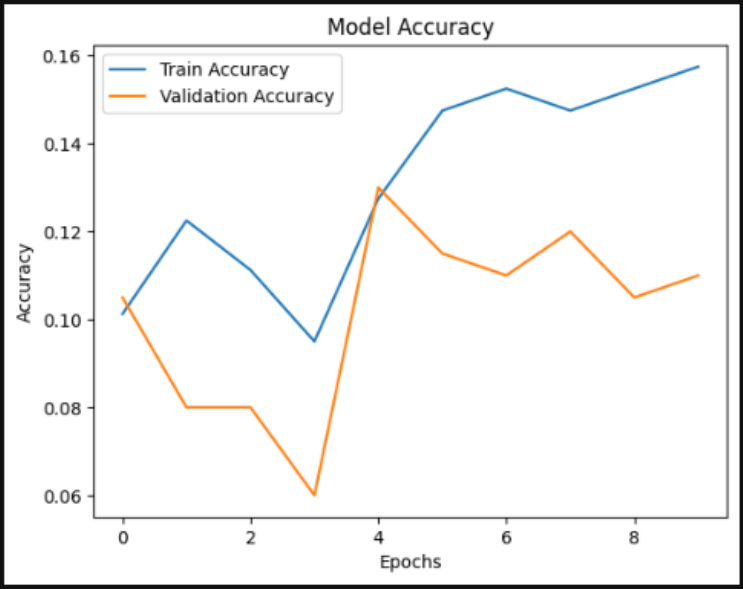
=================================================================
Total params: 2588490 (9.87 MB)
Trainable params: 330506 (1.26 MB)
Non-trainable params: 2257984 (8.61 MB)
_________________________________________________________________
```

[7]:
```
# Train the classifier layers
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Plot training history (optional)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
2024-10-07 03:42:17.824789: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 481689600 exceeds 10% of free system memory.
Epoch 1/10
2024-10-07 03:42:22.079431: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
 1/25 [>.............................] - ETA: 1:40 - loss: 2.6418 - accuracy: 0.1250
2024-10-07 03:42:22.851691: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
 2/25 [=>............................] - ETA: 18s - loss: 2.7284 - accuracy: 0.1406
2024-10-07 03:42:23.628341: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
 3/25 [==>...........................] - ETA: 16s - loss: 2.7208 - accuracy: 0.1250
2024-10-07 03:42:24.365663: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
```

```
 2/25 [=>............................] - ETA: 18s - loss: 2.7284 - accuracy: 0.1406
2024-10-07 03:42:23.628341: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
 3/25 [==>...........................] - ETA: 16s - loss: 2.7208 - accuracy: 0.1250
2024-10-07 03:42:24.365663: W tensorflow/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 156905472 exceeds 10% of free system memory.
25/25 [==============================] - 29s 1s/step - loss: 2.4959 - accuracy: 0.1013 - val_loss: 2.4049 - val_accuracy: 0.1050
Epoch 2/10
25/25 [==============================] - 24s 955ms/step - loss: 2.3505 - accuracy: 0.1225 - val_loss: 2.3353 - val_accuracy: 0.0800
Epoch 3/10
25/25 [==============================] - 25s 1s/step - loss: 2.3610 - accuracy: 0.1112 - val_loss: 2.3960 - val_accuracy: 0.0800
Epoch 4/10
25/25 [==============================] - 27s 1s/step - loss: 2.3497 - accuracy: 0.0950 - val_loss: 2.3231 - val_accuracy: 0.0600
Epoch 5/10
25/25 [==============================] - 24s 986ms/step - loss: 2.3327 - accuracy: 0.1275 - val_loss: 2.3585 - val_accuracy: 0.1300
Epoch 6/10
25/25 [==============================] - 22s 893ms/step - loss: 2.3204 - accuracy: 0.1475 - val_loss: 2.3722 - val_accuracy: 0.1150
Epoch 7/10
25/25 [==============================] - 23s 905ms/step - loss: 2.2954 - accuracy: 0.1525 - val_loss: 2.3394 - val_accuracy: 0.1100
Epoch 8/10
25/25 [==============================] - 23s 903ms/step - loss: 2.3025 - accuracy: 0.1475 - val_loss: 2.3555 - val_accuracy: 0.1200
Epoch 9/10
25/25 [==============================] - 23s 911ms/step - loss: 2.2771 - accuracy: 0.1525 - val_loss: 2.3595 - val_accuracy: 0.1050
Epoch 10/10
25/25 [==============================] - 23s 917ms/step - loss: 2.2786 - accuracy: 0.1575 - val_loss: 2.3410 - val_accuracy: 0.1100
```



[8]:
```python
# Unfreeze some of the last convolutional layers for fine-tuning
for layer in base_model.layers[-20:]:  # Unfreeze last 20 layers; adjust as needed
    layer.trainable = True

# Recompile the model with a lower learning rate for fine-tuning
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Continue training with fine-tuning
fine_tune_history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2)
```

```
Epoch 1/5
25/25 [==============================] - 35s 1s/step - loss: 2.4219 - accuracy: 0.1063 - val_loss: 2.3484 - val_accuracy: 0.0900
```

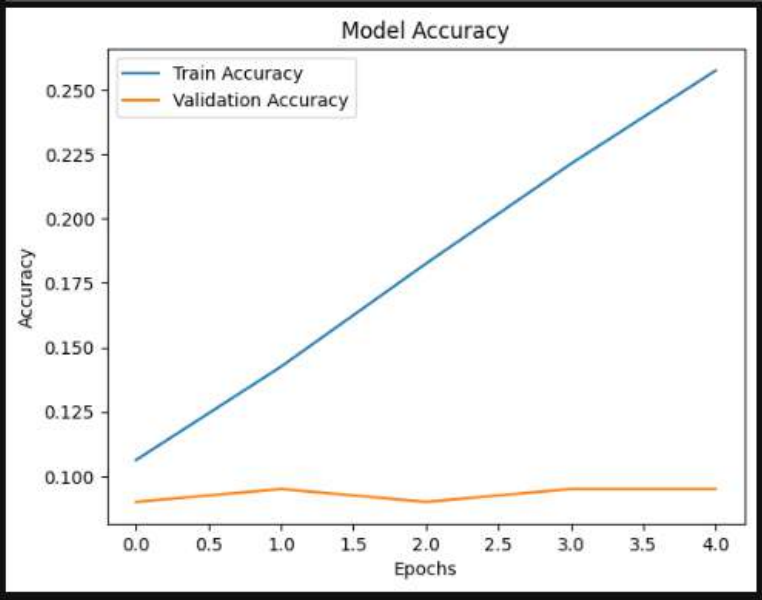
```
    layer.trainable = True

# Recompile the model with a lower learning rate for fine-tuning
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Continue training with fine-tuning
fine_tune_history = model.fit(X_train, y_train, epochs=5, batch_size=32, validation_split=0.2)
```

```
Epoch 1/5
25/25 [==============================] - 35s 1s/step - loss: 2.4219 - accuracy: 0.1063 - val_loss: 2.3484 - val_accuracy: 0.0900
Epoch 2/5
25/25 [==============================] - 29s 1s/step - loss: 2.3002 - accuracy: 0.1425 - val_loss: 2.3540 - val_accuracy: 0.0950
Epoch 3/5
25/25 [==============================] - 29s 1s/step - loss: 2.2386 - accuracy: 0.1825 - val_loss: 2.3591 - val_accuracy: 0.0900
Epoch 4/5
25/25 [==============================] - 29s 1s/step - loss: 2.1972 - accuracy: 0.2212 - val_loss: 2.3634 - val_accuracy: 0.0950
Epoch 5/5
25/25 [==============================] - 31s 1s/step - loss: 2.1624 - accuracy: 0.2575 - val_loss: 2.3641 - val_accuracy: 0.0950
```

[9]:
```
# Plot training history (optional)
plt.plot(fine_tune_history.history['accuracy'], label='Train Accuracy')
plt.plot(fine_tune_history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



[ ]: